IILM UNIVERSITY

C++ Programming Internship

Banking Management System
& Sudoku Solver

SUBMITTED BY

Nilay Sinha

ROLL NO.

25SCS1003003510

PROGRAM

B.Tech CSE
Section 2CSE34

SESSION

2025–29 | 3rd Semester

INSTITUTION

IILM University

AGENDA

What we'll cover

01 **Internship overview & objectives**

CodeAlpha, program goals

02 **Project 1 — Banking Management System**

OOP, file handling, validation

03 **Project 2 — Sudoku Solver**

Recursion & backtracking

04 **Technologies, testing & results**

Concepts applied and verified

05 **Challenges, learnings & what's next**

Outcomes and future scope

INTERNSHIP OVERVIEW

About the internship

Completed under CodeAlpha's C++ Programming Internship program — moving from classroom exercises to practical project development: planning features, writing reusable functions, validating input, handling files, and testing complete applications end to end.

TWO TASKS SELECTED

Task 3 — Sudoku Solver

Recursion, backtracking, constraint checking

Task 4 — Banking Management System

OOP, file handling, transaction management

2

Projects built

16

Test cases verified

1

GitHub repository

GOALS

Internship objectives

Strengthen C++ fundamentals

Syntax, structure, and clean program design

Apply OOP concepts

Classes, objects, and modular design

Improve algorithmic thinking

Logical problem-solving and design

Learn file handling

Persistent storage across program runs

Understand recursion & backtracking

Through a real constraint-based problem

Practice input validation

Robust error handling

Improve debugging & testing

Systematic verification of behaviour

Organize projects for submission

Clean, GitHub-ready code

PROJECT 1

Banking Management System

TASK 4

Console-based
C++ application

A console-based C++ application that simulates core banking operations — built to provide a simple way to create and manage customer accounts while maintaining balances and transaction records.

- Create and manage bank accounts
- Deposit, withdraw, and transfer funds
- Display account details and balance
- Maintain a full transaction history
- Persist data with file storage
- Validate input and block invalid operations

PROJECT 1 · FEATURES

Key features

01

Account Creation

Creates a new account and assigns an account number

02

Deposit

Adds a valid amount to an existing account balance

03

Withdrawal

Withdraws money after checking available balance

04

Fund Transfer

Moves funds between accounts after validation

05

Account Details

Displays account information and current balance

06

Transaction History

Records every important transaction to file

07

Close Account

Allows closure once required conditions are met

08

File Handling

Stores accounts.txt and transactions.txt for persistence

PROJECT 1 · WORKFLOW

How the system works

1 Load

Saved account data is loaded from file at startup

2 Select

User picks an operation from the menu

3 Validate

Amounts, balances, and account existence are checked

4 Execute

Deposit / withdraw / transfer updates the balance

5 Persist

Updated accounts & transactions are saved to file

Deposits increase the balance; withdrawals and transfers only proceed when sufficient funds are available.

PROJECT 2

Sudoku Solver

TASK 3

9×9 grid solved by recursive backtracking

A C++ program that solves a standard 9×9 Sudoku puzzle. Empty cells are represented by 0 — the program searches for valid values and uses backtracking to reach a complete solution.

- Represent the puzzle as a 2D array
- Identify empty cells to fill
- Check row, column & 3×3 sub-grid rules
- Use recursion to explore possibilities
- Backtrack on invalid placements
- Display the solved grid, or report no solution

PROJECT 2 · ALGORITHM

The backtracking loop

1. **Find empty cell**
   Scan the grid for the next cell containing 0
2. **Try a number**
   Attempt 1 through 9 in that cell
3. **Check isSafe()**
   Validate against row, column & 3×3 box
4. **Place & recurse**
   If safe, place it and solve the rest
5. **Backtrack**
   If it fails later, reset to 0 and try the next number

"Put a number. Try solving the rest. If it works, keep it. If not, erase it and try another."

— the core backtracking logic in one sentence

PROJECT 2 · STRUCTURE

Core functions & constraints

MAIN FUNCTIONS

`findEmptyCell()`

Finds an empty position in the grid

`isSafe()`

Checks row, column & 3×3 sub-grid rules

`solveSudoku()`

Recursion + backtracking to solve the puzzle

`printGrid()`

Displays the grid in a readable format

SUDOKU CONSTRAINTS

Each row contains 1–9 without repetition

Each column contains 1–9 without repetition

Each 3×3 sub-grid contains 1–9 without repetition

Handles both solvable and unsolvable puzzles correctly

ACROSS BOTH PROJECTS

Technologies & concepts used

CONCEPT / TECHNOLOGY	APPLICATION IN PROJECTS
C++	Primary language used for both projects
Object-Oriented Programming	Organizing banking accounts & operations
Classes & Objects	Modeling accounts and their behaviour
Functions	Breaking programs into reusable tasks
File Handling	Storing account & transaction data
Arrays / Data Structures	Representing the Sudoku grid
Recursion & Backtracking	Solving Sudoku systematically
Input Validation	Preventing invalid values & operations

VERIFICATION

Testing & results

PROJECT	TEST CASE	EXPECTED RESULT
Banking	Create a new account	Account created with unique number
Banking	Deposit a valid amount	Balance increases correctly
Banking	Withdraw more than balance	Transaction rejected
Banking	Transfer between valid accounts	Amount deducted & credited correctly
Banking	Close account with balance	Closure rejected
Sudoku	Provide a solvable puzzle	Solved grid displayed
Sudoku	Provide an unsolvable puzzle	"No solution" reported
Sudoku	Check invalid placement	Number rejected per Sudoku rules

ALONG THE WAY

Challenges & solutions

Managing account data

Structured the app using classes and organized functions

Persistent storage

Used file handling to save & reload account/transaction data

Preventing invalid operations

Added checks for existence, positive amounts, balance limits

Understanding recursion

Broke Sudoku solving into small recursive steps

Backtracking logic

Reset earlier choices whenever a path failed

Debugging

Tested operations individually and fixed logical errors through repeated runs

TAKEAWAYS

Learning outcomes

- Improved understanding of C++ syntax and program structure
- How recursion works in a practical algorithm
- Practical experience with object-oriented programming
- The backtracking approach for constraint-based problems
- Designing a menu-driven console application
- Better debugging and testing skills
- Stronger grasp of file input/output
- Organizing projects for GitHub submission

WRAPPING UP

Conclusion & future scope

CONCLUSION

The internship bridged theoretical programming concepts and practical implementation. The banking project strengthened OOP, file handling and validation; the Sudoku project strengthened recursion, backtracking and algorithmic thinking.

FUTURE SCOPE

- Add a graphical interface to both applications
- Use a database instead of text files
- Add secure authentication & role-based access
- Add Sudoku difficulty levels & puzzle generation
- Improve documentation & automated testing

Thank You

CodeAlpha C++ Programming Internship — Nilay Sinha

Repo: CodeAlpha_SudokuSolver • GitHub • LinkedIn • IILM University